MODULE – 2

- **WHAT IS JDBC?**
  - **JDBC** stands for **Java Database Connectivity**.
  - It is a **Java API (Application Programming Interface)** that allows Java applications to interact with a wide range of **databases** using **SQL (Structured Query Language)**.
- **Importance of JDBC in Java Programming**
  - JDBC (Java Database Connectivity) plays a crucial role in Java programming, especially for applications that involve data storage, retrieval, and manipulation.
    1. Database Connectivity
    2. Platform Independence
    3. Standardized Access
    4. Supports All SQL Operations
    5. Enables Enterprise Applications
    6. Flexible and Extensible
    7. Enables Dynamic Applications
    8. Transaction Management
    9. Integration with Java Technologies
- **JDBC Architecture: Driver Manager, Driver, Connection, Statement, and Result Set**
  1. **Driver Manager**
     - Manages a list of database drivers and establishes a connection between the Java application and the database.
     - Connection con = DriverManager.getConnection(...);
  2. **Driver**
     - A JDBC Driver is a software component that implements the JDBC interfaces for a particular database.

Types of Drivers:

Type	Name	Description
Type 1	JDBC-ODBC Bridge	Connects using ODBC driver (outdated).
Type 2	Native-API Driver	Converts JDBC to native database calls.
Type 3	Network Protocol Driver	Sends JDBC calls through a middleware server.
Type 4	Thin Driver (Pure Java)	Direct communication with the database in Java – **most common today**.

Example:
"com.mysql.cj.jdbc.Driver" : is a Type 4 driver for MySQL.

3. **Connection**
   - Represents a session with the database.
   - Obtained via the DriverManager.
   - Connection con = DriverManager.getConnection(url, username, password);
4. **Statement**
   - Used to **send SQL commands** (queries or updates) to the database.

Types of Statements:

Type	Class	Use
Regular	Statement	For static SQL (no parameters).
Parameterized	PreparedStatement	For SQL with input parameters.
Callable	CallableStatement	For calling stored procedures.

Example:

```
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM users");
```

5. **ResultSet**
   - A table of data representing the result of a SQL query.

```
while(rs.next()) {
   System.out.println(rs.getString("username"));
}
```

•